

Lab Manual — Week 6

Concurrency & Distributed Systems — Lebanese University, Faculty of Engineering

Topic: Parallelism and Fork/Join

Learning Objectives

By the end of Week 6, you will understand:

1. **Parallel decomposition:** How one CPU-bound computation can be split into independent subtasks.
2. **Fork/Join:** How `ForkJoinPool` and `RecursiveTask` structure divide-and-conquer work.
3. **Work stealing:** Why idle workers can steal queued subtasks from busy workers.
4. **Granularity:** How threshold selection controls task size and overhead.
5. **Break-even:** How to measure the minimum input size where Fork/Join can beat sequential execution.
6. **Parallel stream pitfalls:** Why `parallel()` can be slower than a primitive sequential loop.
7. **Blocking hazards:** Why blocking I/O inside the common `ForkJoinPool` can starve CPU-bound work.

Required Course Artifacts

This lab is both a coding lab and a measurement lab. You must submit evidence, not only code.

- **Concurrency scorecard:** complete the Week 6 `CONCURRENCY_SCORECARD.md`.
- **Measurement table:** record sequential and Fork/Join timings for multiple sizes.
- **Break-even paragraph:** explain the first size where Fork/Join wins, or why no win appears in your run.
- **Parallel-stream diagnosis:** name the four slowdown causes from Exercise 3.
- **Caveat statement:** include hardware, JDK, threshold, and input-size caveats for any performance claim.

Lab Structure

Week 6 is organized into 3 parts:

Part	Topic	Demos + Exercise
Part 1	Sequential baseline and Fork/Join model	Demo01–Demo03, Ex1
Part 2	Granularity and break-even measurement	Demo04–Demo05, Ex2
Part 3	Parallel streams and common-pool pitfalls	Demo06–Demo07, Ex3

1 Part 1: Fork/Join Model

1.1 Demo01: SequentialArraySum

Goal: Establish the simplest correct implementation.

Logic:

- Iterate over the array.
- Accumulate values into a local `long`.
- Return the final total.

Key Insight: The sequential implementation is the correctness reference. A parallel implementation that is faster but wrong is not an optimization.

1.2 Demo02: ForkJoinArraySum

Goal: Show a correct `RecursiveTask` implementation with a threshold.

Logic:

- If the range length is at or below the threshold, sum that range sequentially.
- Otherwise split the range into two halves.
- Fork one side, compute the other side directly, then join.
- Combine returned values instead of mutating shared state.

Code Pattern:

```
if (length <= threshold) {  
    return sequentialSum(values, start, end);  
}  
  
SumTask left = new SumTask(values, start, middle, threshold);  
SumTask right = new SumTask(values, middle, end, threshold);  
left.fork();  
long rightResult = right.compute();  
long leftResult = left.join();  
return leftResult + rightResult;
```

Key Insight: Fork/Join works best when subtasks are independent and side-effect free.

1.3 Demo03: ForkJoinPoolContext

Goal: Make the common pool visible.

Logic:

- Read `ForkJoinPool.commonPool().getParallelism()`.
- Read `Runtime.getRuntime().availableProcessors()`.
- Compare pool parallelism with available CPU resources.

Key Insight: The common pool is shared process-wide. Hidden sharing matters when parallel streams or other code also use it.

1.4 Exercise 1: ForkJoinSum

Goal: Implement a Fork/Join sum that matches the sequential result.

Requirements:

- Use a sequential base case for small ranges.
- Split ranges larger than the threshold.
- Fork one subtask, compute the other, then join.
- Return the sum of the two subtask results.

2 Part 2: Granularity and Break-Even

2.1 Demo04: BreakEvenProbe

Goal: Measure sequential vs Fork/Join execution for the same workload.

Logic:

- Create an array of a given size.
- Run the sequential sum and measure elapsed nanoseconds.
- Run the Fork/Join sum with the same data and threshold.
- Record whether both implementations returned the same result.

Key Insight: Measurement without a correctness check is not enough. You must prove both versions computed the same work.

2.2 Demo05: GranularityCost

Goal: Show how threshold changes the number of leaf tasks.

Logic:

- Recursively split the input size until each piece is at or below the threshold.
- Count how many leaf tasks are produced.
- Smaller thresholds create more leaf tasks.

Key Insight: Tiny tasks create overhead. Large tasks reduce parallelism. The threshold balances these costs.

2.3 Exercise 2: BreakEvenReport

Goal: Produce a measurement report that identifies where Fork/Join can win.

Requirements:

- Accept an array of input sizes and a threshold.
- Preserve input order in the returned rows.
- For each size, measure sequential and Fork/Join execution.
- Return the first size where Fork/Join is faster, or -1.
- Include an explanation that names coordination overhead.

Suggested Table:

Size	Threshold	Sequential ns	Fork/Join ns	Same result
1,000	128			
10,000	128			
100,000	128			
1,000,000	128			

Break-Even Paragraph Template:

On this machine, with JDK ____ and threshold ____, Fork/Join first beat sequential at size _____. Before that point, split, scheduling, and join overhead dominated the useful work. This result may change with CPU count, JVM warm-up, input size, and threshold.

3 Part 3: Parallel Streams and Common-Pool Pitfalls

3.1 Demo06: ParallelStreamAntiExample

Goal: Demonstrate that a parallel stream is not automatically faster.

Logic:

- Compute a numeric sum using a primitive sequential stream.
- Compute the same result using a deliberately boxed parallel stream.
- Name the costs that can make the parallel version slower.

Code Pattern:

```
long sequential = IntStream.rangeClosed(1, size)
    .asLongStream()
    .sum();

long parallel = IntStream.rangeClosed(1, size)
    .boxed()
    .parallel()
    .mapToLong(Integer::longValue)
    .sum();
```

Key Insight: `parallel()` adds machinery. If each element has little work, the machinery can dominate the computation.

3.2 The Four Required Slowdown Causes

1. **Boxing overhead:** primitive values become objects.
2. **Lambda allocation or invocation overhead:** pipeline stages add work.
3. **Thread contention:** common-pool workers are shared.
4. **Splitting overhead:** the stream must partition and combine work.

3.3 Demo07: BlockingInCommonPoolRisk

Goal: State the main `ForkJoinPool` liveness hazard.

Logic:

- The common pool has limited worker parallelism.
- Blocking workers cannot execute CPU-bound subtasks.
- Parallel streams and `Fork/Join` tasks may share the same pool.

Key Insight: `Fork/Join` is for CPU-bound decomposition. Blocking I/O belongs in a different execution policy.

3.4 Exercise 3: ParallelStreamDiagnosis

Goal: Implement matching sequential and parallel computations, then diagnose why the parallel version can be slower.

Requirements:

- Compute $1 + 2 + \dots + \text{size}$ sequentially.
- Compute the same result with a deliberately boxed parallel stream.
- Return exactly the four required slowdown causes.

- Avoid claiming that parallel streams are always faster.

Week 6 Summary

- Fork/Join is a divide-and-conquer model for CPU-bound work.
- The sequential baseline is mandatory for correctness and performance comparison.
- The threshold controls granularity; there is no universal best threshold.
- Break-even measurement must include correctness, repeated observations, and caveats.
- Parallel streams are convenient but share the common pool and can be slower.
- Blocking I/O inside `ForkJoinPool` is a worker starvation risk.

Completion Checklist

- ☐ `DemoSmokeTest` passes.
- ☐ `studentCheck` passes after implementing all exercises.
- ☐ `Ex1_ForkJoinSum` uses a sequential base case and recursive split.
- ☐ `Ex2_BreakEvenReport` returns ordered rows and an overhead explanation.
- ☐ `Ex3_ParallelStreamDiagnosis` lists all four slowdown causes.
- ☐ The scorecard is complete.
- ☐ The break-even paragraph includes hardware/JDK/threshold caveats.

Bridge to Week 7

Week 6 is about splitting CPU-bound computation. Week 7 moves toward async design: composing tasks that may wait, time out, fall back, or finish in a different order. Do not confuse these models. Fork/Join is not a replacement for timeout-aware async workflows.